Zadanie 3. Zaawansowane metody lokalnego przeszukiwania

Inteligentne Metody Optymalizacji

Maciej Szymaniak, 155830
Piotr Szymaczek, 155889

27 kwietnia 2026

1 Opis zadania

Rozważamy zmodyfikowany problem komiwojażera. Dany jest zbiór n wierzchołków z symetryczną macierzą odległości d_{ij} oraz zyskiem p_i za odwiedzenie wierzchołka i. Należy wybrać podzbiór wierzchołków i ułożyć zamkniętą trasę przechodzącą przez wybrane wierzchołki, maksymalizując funkcję celu:

$$f(\mathcal{C}) = \sum_{i \in \mathcal{C}} p_i \; - \sum_{(i,j) \in \mathcal{C}} d_{ij}$$

Praca zawiera porównanie następujących algorytmów:

- **SLS** (Steepest Local Search) – pełne przeszukiwanie sąsiedztwa stanowiące punkt odniesienia.
- **LM** (Move List) – wersja wykorzystująca uporządkowaną listę ruchów poprawiających oraz aktualizację inkrementalną.
- **CM** (Candidate Moves) – ograniczenie sąsiedztwa do k (testowane k=10) najbliższych sąsiadów. Ruchy są rozważane tylko wtedy, gdy wprowadzają krawędź kandydacką.
- **Regret** – zaawansowana heurystyka konstrukcyjna z mechanizmem żalu, stanowiąca bazowy punkt odniesienia.

Kod źródłowy dostępny pod adresem:
`https://github.com/masz00/Intelligent-Optimization-Methods/LocalSearchAdvanced`
Rozwiązania zweryfikowane za pomocą SolutionCheckera: *TAK*.

2 Pseudokody algorytmów

Algorithm 1: SLS z listą ruchów (Move List - LM)

```
Wejście: Rozwiązanie początkowe x, lista wierzchołków V, macierz odległości D
Wyjście: Zoptymalizowana trasa x

PQ ← Kolejka priorytetowa typu Max-Heap przechowująca ruchy poprawiające;
pos ← Tablica pomocnicza: pos[i] to indeks wierzchołka i w trasie x;

dla każdego krawędź (a, b) ∈ x wykonaj
    Znajdź wszystkie ruchy ADD(u) oraz 2-OPT(a, b, c, d) poprawiające wynik;
    Wstaw ruchy do PQ z priorytetem równym ich wartości poprawy (Δ);
koniec
dla każdego wierzchołek u ∈ x wykonaj
    Oceń ruch REMOVE(u) i jeśli Δ > 0, wstaw do PQ;
koniec

dopóki PQ nie jest pusta wykonaj
    m ← PQ.pop() jeżeli ruch m jest nadal aplikowalny ORAZ Δ > 0 to
        Wykonaj ruch m(x) i zaktualizuj tablicę pos;

        dla każdego nowo powstała krawędź (i, j) ∈ x wykonaj
            Znajdź nowe ruchy poprawiające związane z (i, j) i dodaj do PQ;
        koniec
        dla każdego wierzchołek v sąsiadujący ze zmianą wykonaj
            Oceń nowe możliwości REMOVE(v) i dodaj do PQ;
        koniec
    koniec
koniec
zwróć x
```

Algorithm 2: SLS z ruchami kandydackimi (Candidate Moves - CM)

Wejście: Rozwiązanie początkowe x, lista wierzchołków V, liczba sąsiadów $k = 10$
Wyjście: Zoptymalizowana trasa x

```
dla każdego v ∈ V wykonaj
    N(v) ← posortowana lista k najbliższych wierzchołków do v (według D);
koniec
dopóki znaleziono poprawę wykonaj
    best_move ← NONE,   Δ_best ← 0;
    dla każdego wierzchołek v na trasie x wykonaj
        v_next ← następnik v w trasie;
        dla każdego sąsiad u ∈ N(v) wykonaj
            jeżeli u ∉ x to
                Δ ← zysk(u) − (D_{v,u} + D_{u,v_next} − D_{v,v_next});
            koniec
            w przeciwnym razie
                Δ ← (D_{v,v_next} + D_{u,u_next}) − (D_{v,u} + D_{v_next,u_next});
            koniec
            Zaktualizuj best_move, jeśli Δ > Δ_best;
        koniec
    koniec
    dla każdego wierzchołek w na trasie x wykonaj
        w_prev, w_next ← poprzednik i następnik w;
        jeżeli krawędź (w_prev, w_next) jest kandydacka (tj. w_next ∈ N(w_prev)) to
            Δ ← (D_{w_prev,w} + D_{w,w_next}) − D_{w_prev,w_next} − zysk(w);
            Zaktualizuj best_move, jeśli Δ > Δ_best;
        koniec
    koniec
    jeżeli Δ_best > 0 to
        Wykonaj best_move(x);
    koniec
koniec
zwróć x
```

3 Wyniki eksperymentów

Eksperyment przeprowadzono, wykonując 100 uruchomień każdego algorytmu. Dla metod lokalnego przeszukiwania rozwiązaniem startowym był losowy podzbiór wierzchołków obejmujący 50% instancji.

3.1 Instancja TSPA

Algorytm	Wynik Avg (Min - Max)	Avg Time [ms]
SLS	6341 (4277 - 7577)	4.95
LM	5844 (2870 - 7715)	8.52
CM	5950 (3866 - 7408)	5.40
Regret	4809 (3840 - 6202)	2.67

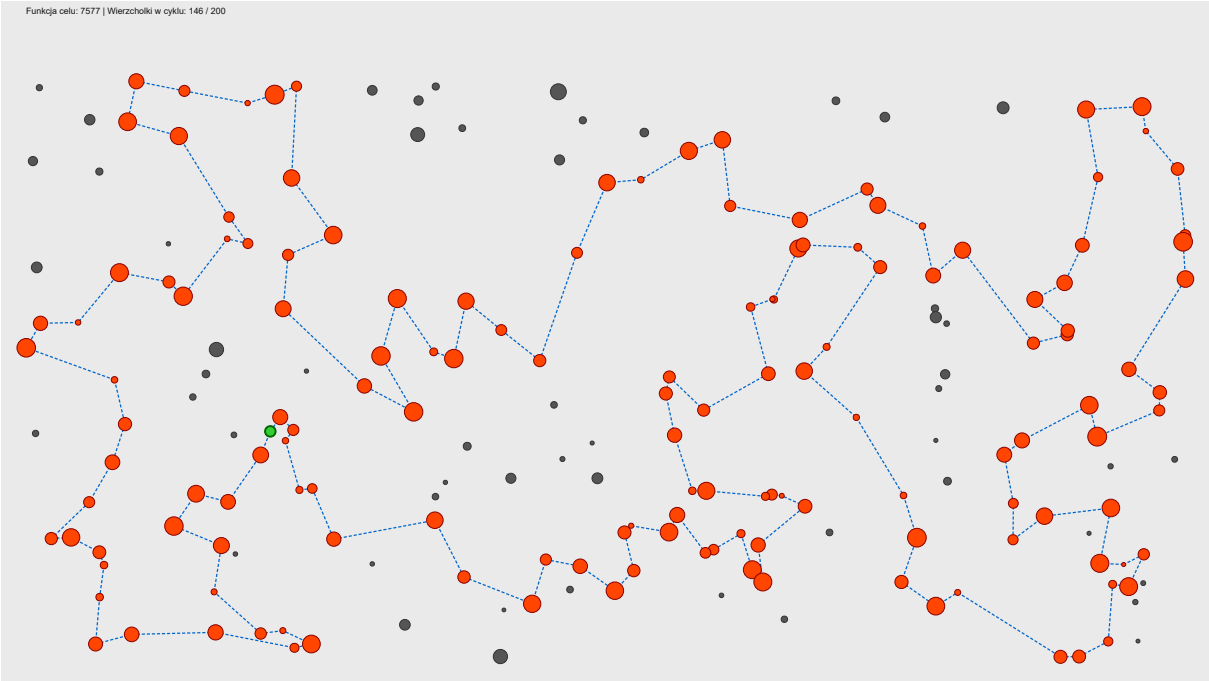
Tabela 1: Wyniki dla instancji TSPA (100 uruchomień)

3.2 Instancja TSPB

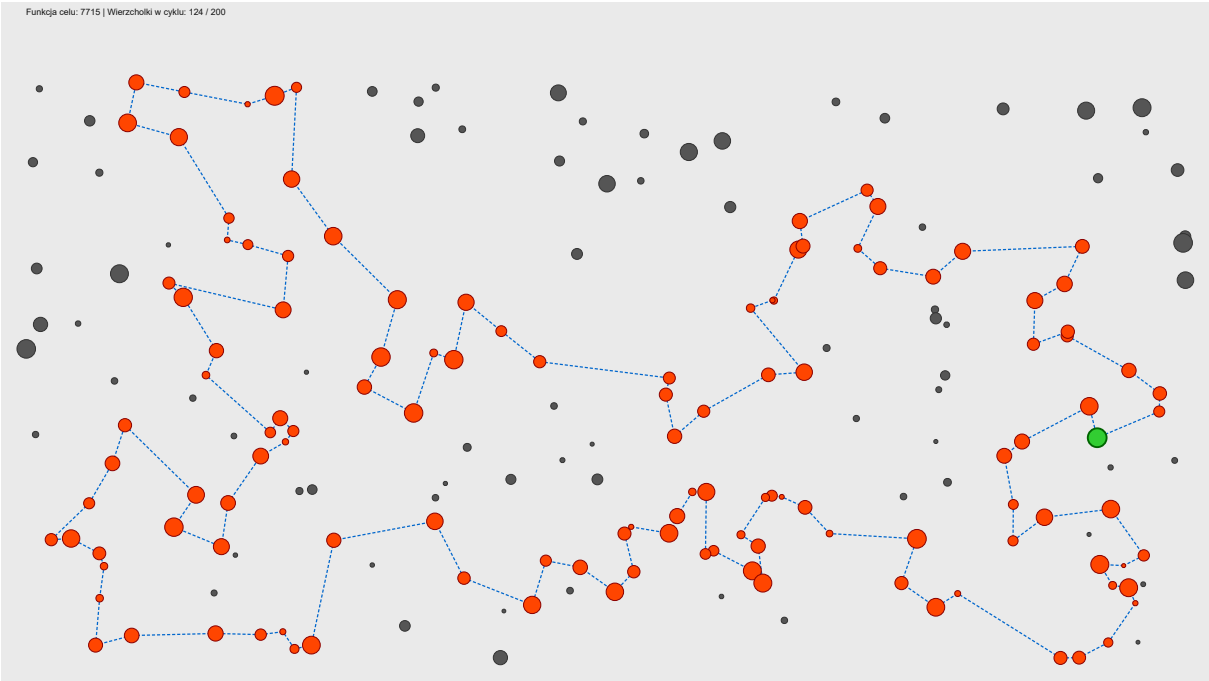
Algorytm	Wynik Avg (Min - Max)	Avg Time [ms]
SLS	17864 (15053 - 19066)	6.27
LM	17237 (12364 - 19444)	9.26
CM	17559 (14551 - 19379)	6.00
Regret	17570 (16462 - 18324)	2.54

Tabela 2: Wyniki dla instancji TSPB (100 uruchomień)

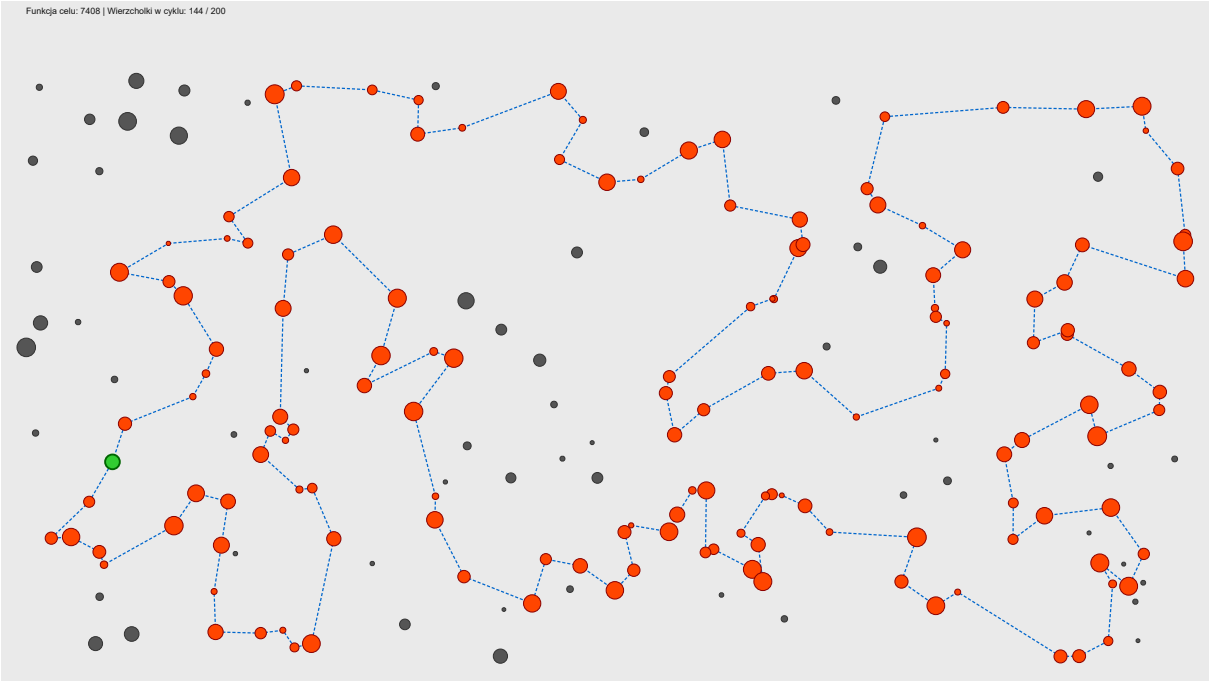
4 Wizualizacje najlepszych rozwiązań



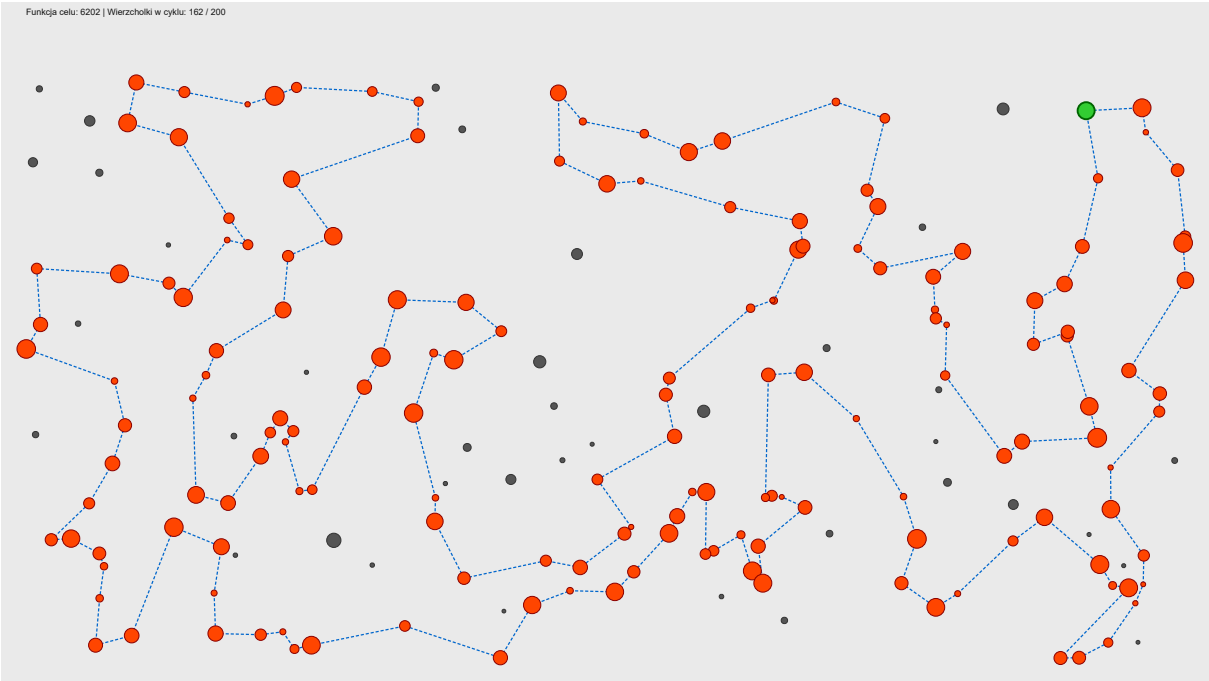
Rysunek 1: TSPA - Metoda SLS



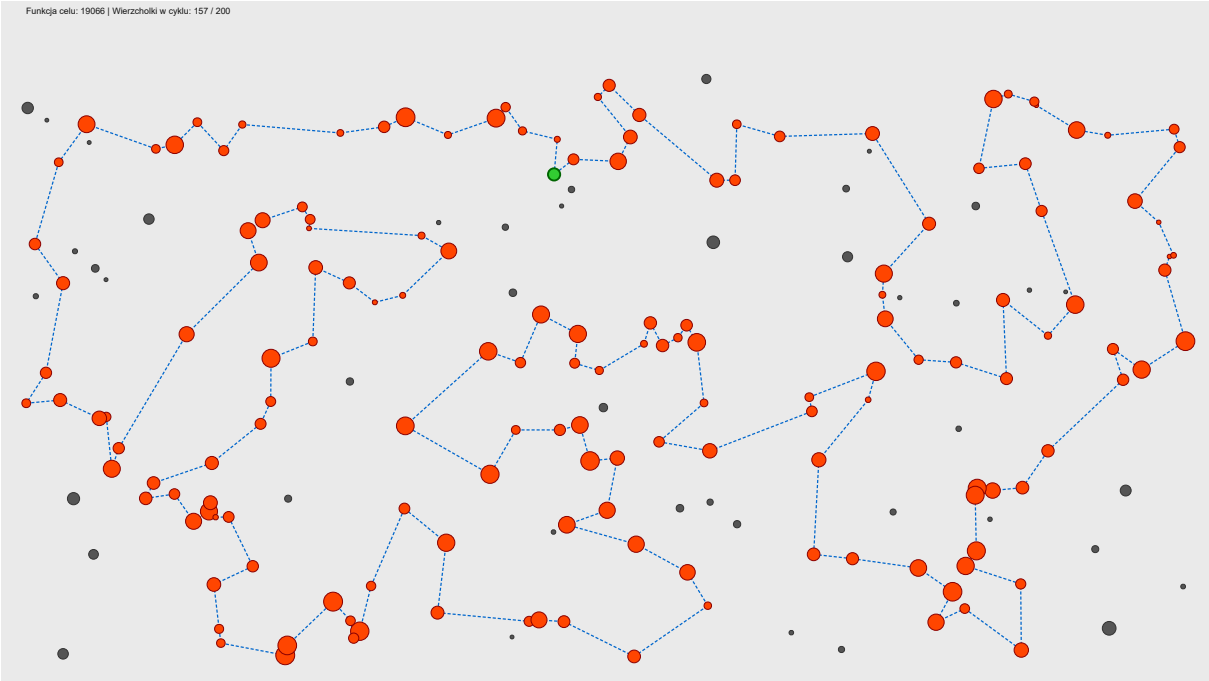
Rysunek 2: TSPA - Metoda LM



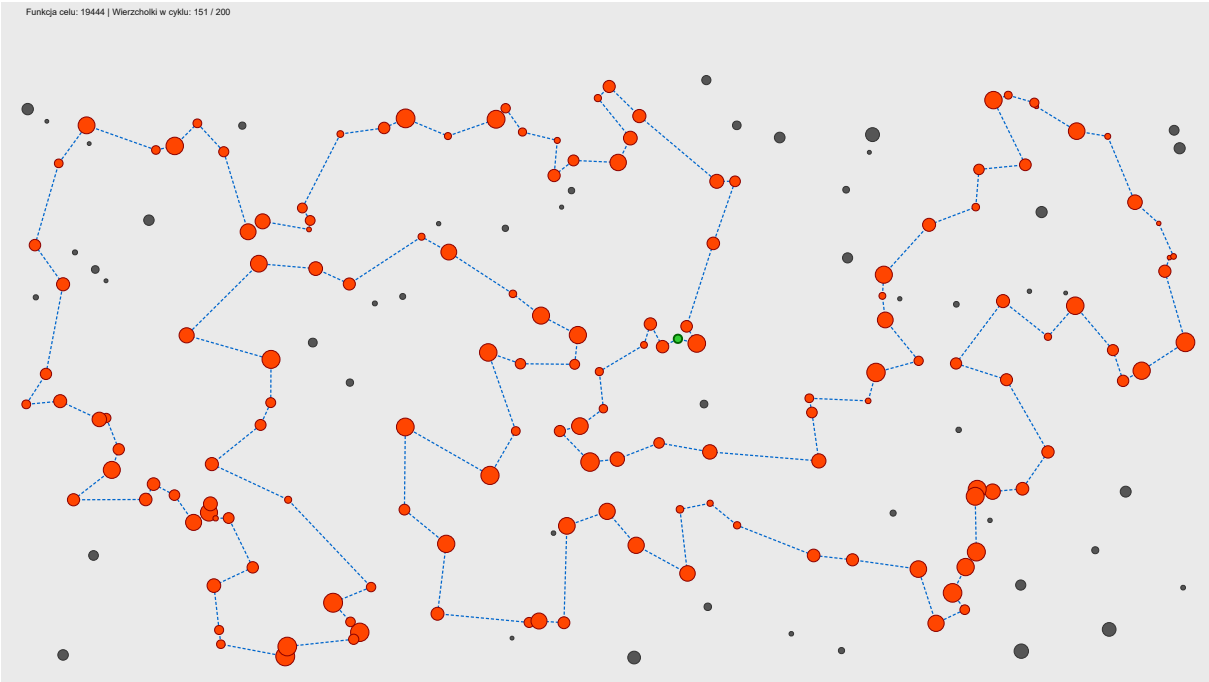
Rysunek 3: TSPA - Metoda CM



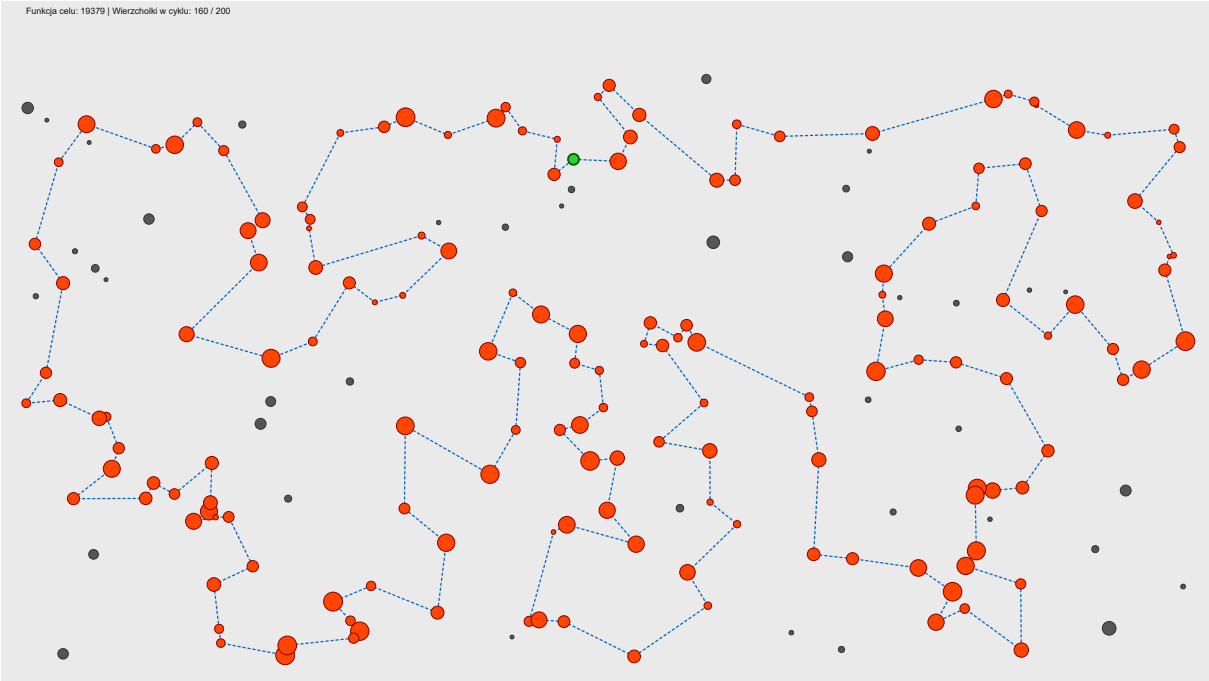
Rysunek 4: TSPA - Heurystyka Regret



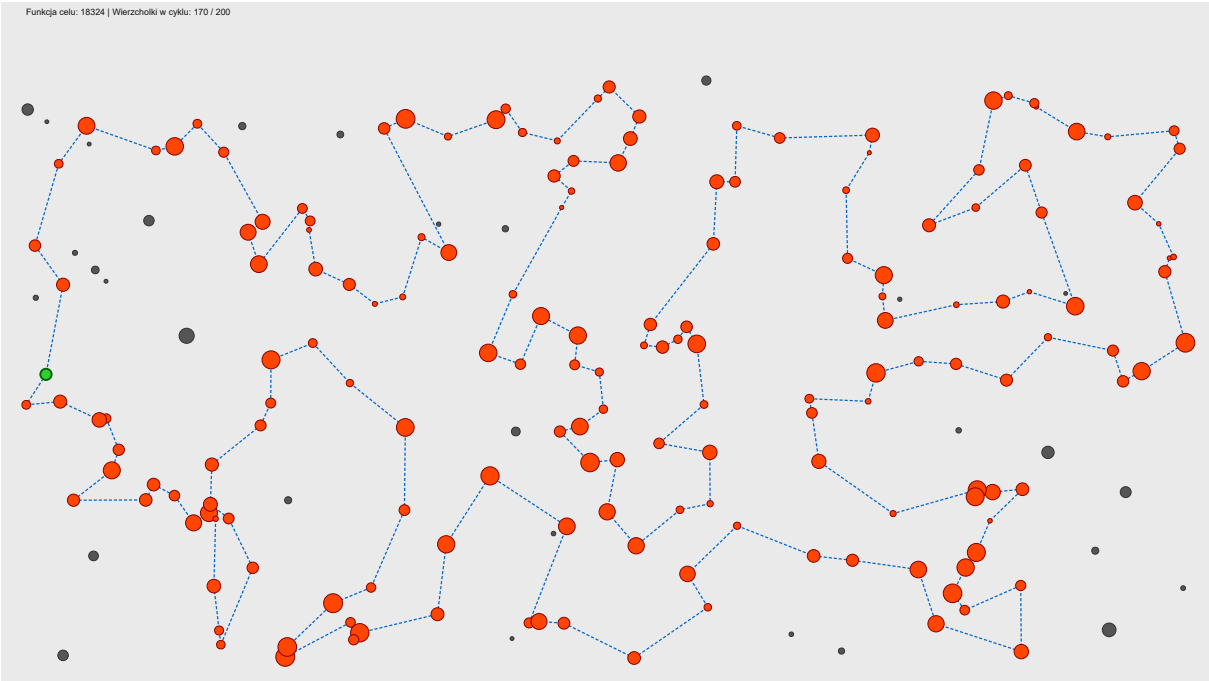
Rysunek 5: TSPB - Metoda SLS



Rysunek 6: TSPB - Metoda LM



Rysunek 7: TSPB - Metoda CM



Rysunek 8: TSPB - Heurystyka Regret

5 Wnioski

1. **Przewaga Local Search**: Wszystkie metody lokalnego przeszukiwania (SLS, LM, CM) dały generlanie lepsze wyniki niż heurystyka Regret z poprzedniego zadania. To pokazuje, że nawet startując z losowego rozwiązania, warto poświęcić czas na optymalizację ruchami 2-OPT i ADD/REMOVE.

2. **Analiza czasu LM**: W naszych testach LM wyszedł nieco wolniej niż zwykły SLS. Wynika to z faktu, że dla małych instancji (tutaj 200 punktów) obsługa kolejki priorytetowej i dodatkowych tablic zajmuje procesorowi więcej czasu niż szybkie przeliczenie zmian w zwykłej pętli. Przewaga LM prawodopodonie byłaby widoczna przy większych instancjach.

3. **Skuteczność Candidate Moves**: Metoda CM (testowane tylko k = 10 sąsiadów) to świetny kompromis. Sprawdzamy tylko ułamek wszystkich możliwych ruchów, a wyniki mamy prawie tak samo dobre jak w pełnym SLS.